Course Organisation

Summary

- **CS-328**, Prof. Wenzel Jakob (Realistic Graphics Lab, RGL).
- **Lecture** : Thursday, CE13, 10:15-12:00 (concepts, live examples, general Q&A).
- **Exercise session** : Thursday, BC07/08, 16:15-18:00 (first 3 weeks : tutorials, then homework Q&A; ignore the INF3 room).
- TAs : Dongsu Zhang, Ekrem Yilmazer (PhD); Jordan Warne, Felix Burchardt, Arthur Herbette (AEs).

Key formulas

Grading

- 4 homeworks during the semester : **10%** of the final grade.
- Pen & paper exam during the January exam session : **90%** of the final grade.
- Optional practice midterm : graded but 0% weight.
- Homework and exam graded on a 1-6 scale, then averaged with these weights (mapping from raw percentages generally uses a curve).

Homeworks

1. Basics : NumPy, SciPy, Matplotlib, floating point arithmetic, error analysis, first steps with a Monte Carlo method.
2. Processing a RAW image : demosaic & process RAW Bayer grid data from a Canon DSLR, solve a least squares problem from "Native Camera" to human (sRGB) color space.
3. Inverse kinematics : simple IK "chain" (a worm body being pulled), SVD to solve a rank-deficient linear system.
4. Autodiff & machine learning : build a tiny ML framework ("MyTorch") and train a tiny neural network.

Plus 6-7 ungraded exercise sheets. No text book (optional readings); recommended : *Essence of Linear Algebra* (3Blue1Brown).

Warning

- Late policy : 25% penalty for each extra started late day (no points after 4 days).
- Survey course : many methods, none in depth. Not a programming language course.
- Any modern LLM can solve all problems of this course, but reviewing LLM output does not work to learn : "to learn, you must struggle". Huge drop in exam performance in 2026; the exam will not be curved up this year.

Python : reference version **3.14** (standard Python or Conda); an interpreted language with an unusual indentation-based syntax. "Jupyter notebook" means **Jupyter Lab**.

What is a Numerical Method ?

Key idea

Computer graphics, computer vision, machine learning, computational photography, weather simulation, geometric information systems… : **all of them have some kind of numerical method at their core**, running on number crunchers (CPUs, GPUs).

Example

- **Rendering** : visual effects $\simeq$ solving integrals, pixel color $= \int \text{light}\, \mathrm{d}x$.
- **Fluid simulation** : to compute the next frame ("timestep"), solve an enormous linear system $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ (HUGE matrix).
- **Character animation** : attaching a virtual body to a skeleton using optimization, $\mathbf{x} = \mathbf{A}^{+}\mathbf{b}$ (pseudoinverse).
- **Computer vision** : quickly process images using learned convolutions ("CNNs"); a convolution is a particular type of linear transformation.
- **Machine learning** : computing derivatives of software, $\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x})$ where f is a computer program.

Definition

CS-328
About the **computer implementation** of abstract concepts from MATH-111 (Linear Algebra : $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$) and MATH-101/106 (Analysis : $\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x})$, $y = \int_\Omega f(x)\,\mathrm{d}x$).
Typical computer science works with $\mathbb{N}$ (graph theory, databases, network protocols, cryptography); numerical methods work with $\mathbb{R}^n$.

Summary

Many layers (top to bottom)

- Applications
- Frameworks : NumPy, PyTorch, TensorFlow, JAX
- Optimization algorithms : gradient descent, Newton's method in N dimensions, Broyden's method, Gauss-Newton, Levenberg-Marquardt
- Matrix factorizations : the "Big 3" LU, QR, SVD
- Compilation : of floating point math, just-in-time (JAX, XLA, etc.)
- Floating point arithmetic : history, IEEE-754, error propagation (*today*)
- Processor architecture : superscalar processors, vectorization, memory hierarchy, shared memory (GPUs)

Number Systems

Key idea

π has infinitely many digits : we cannot store it, and there are infinitely many numbers of this type between any two numbers. The compromise : **the program will no longer compute the correct answer**.

Definition

INTEGERS

- Positive : $11000101_2 = 2^7 + 2^6 + 2^2 + 2^0 = 197$.
- Signed (two's complement), top bit weighs -2^7 : $11000101_2 = -2^7 + 2^6 + 2^2 + 2^0 = -59$.

Type	Min	Max
`uint8`	0	255
`int8`	-128	127
`uint16`	0	65535
`int16`	-32768	32767
`uint32`	0	4294967295
`int32`	-2147483648	2147483647
`uint64`	0	18446744073709551615
`int64`	-9223372036854775808	9223372036854775807

Definition

FIXED POINT ARITHMETIC
Allow negative exponents : $1100.0101_2 = 2^3 + 2^2 + 2^{-2} + 2^{-4} = 12.3125$.

Warning

— Drawback 1 : big numbers require a lot of storage (4×10^{80} : ∼268 bits).
— Drawback 2 : growth of the exponent range during operations :

$$\Big(\sum_{i=-k}^{k} a_i 2^i\Big)\Big(\sum_{i=-k}^{k} b_i 2^i\Big) = a_{-k}b_{-k}2^{-2k} + \cdots + a_k b_k 2^{2k}$$

Definition

RATIONAL NUMBERS
$\mathbb{Q} = \{a/b : a, b \in \mathbb{Z}\}$: exact representation, $a/b + c/d = (ad + cb)/bd$. Generally not a good thing (numerators and denominators grow).

Floating Point Numbers

Different scales, from the Planck length (10^{-33} cm) to the universe : we need numbers whose resolution follows their magnitude.

Definition

FLOATING POINT NUMBER (“baby version”)

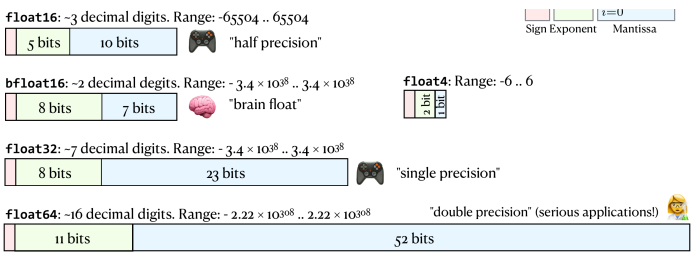
$$\underbrace{\pm}_{\text{sign}} \underbrace{2^E}_{\text{exponent}} \underbrace{\sum_{i=0}^{n} 2^{-i} A_i}_{\text{mantissa}}$$

Example

$$1.0 = +2^0(2^{-0}) = +2^0\, 1.00000_2$$
$$0.5 = +2^{-1}(2^{-0}) = +2^{-1}\, 1.00000_2$$
$$-0.75 = -2^{-1}(2^{-0} + 2^{-1}) = -2^{-1}\, 1.10000_2$$
$$3.14 \approx +2^1(2^{-0} + 2^{-1} + 2^{-4} + 2^{-7}) = +2^1\, 1.1001001_2$$

The leading digit is always 1 : no need to store it.

Standard floating point formats (sign | exponent | mantissa)

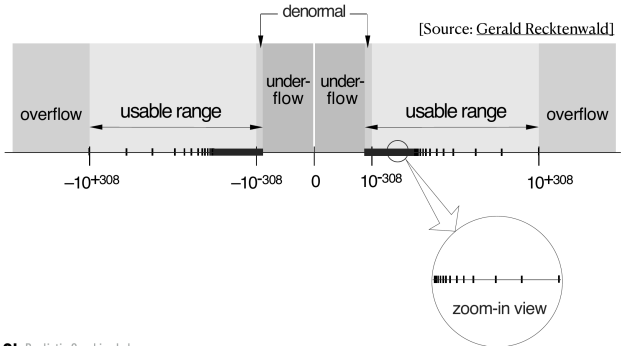


Warning

“Please don’t memorize these numbers for the exam.”

Key idea

Number resolution changes with magnitude : representable numbers are dense near zero and sparse for large magnitudes; overflow beyond $\pm 10^{308}$, underflow near 0 (float64).



The IEEE 754 Standard

Problems with the baby version : how to represent zero ? What if the number becomes too big, or too small, during a calculation ? What if the calculation is nonsensical ? What if the result cannot be represented exactly ?

Definition

IEEE 754 (William Kahan ; published in 1985, developed for many years)
Super-ambitious proposal at the time. Behavior is **fully specified** : every arithmetic operation produces a result, whether it is mathematically expected or not, and exceptional operations raise a signal.

Summary

— **Zero** : easy, just don’t store the leading one. Zero is **signed** in IEEE754.
— **Too big** : return the special number “infinity” (positive or negative).
— **Too small** : gradual underflow through **denormalized numbers**.
— **Nonsensical** : return the special number **NaN** (Not a Number). NaN is infectious.
— **Not representable** : **Banker’s rounding** (round-to-nearest-tie-to-even) by default, other modes available.

Key idea

Major innovations

— **Accuracy guarantees** : elementary operations ($+, -, *, /, \sqrt{}$) are done in infinite precision and then *rounded* to a representable number.
— **Denormalized numbers** ($+2^E \cdot 0.xxx$ instead of $+2^E \cdot 1.xxx$) : fill the gap around zero, enable gradual underflow. Most controversial feature of IEEE 754 : handled badly by current processors, can make a calculation 100-1000× slower.

Special Cases and Danger Zone

$*$	-Inf	-0	0	Inf	NaN
-Inf	Inf	NaN	NaN	-Inf	NaN
-0	NaN	0	-0	NaN	NaN
0	NaN	-0	0	NaN	NaN
Inf	-Inf	NaN	NaN	Inf	NaN
NaN	NaN	NaN	NaN	NaN	NaN

Common sense : $(-\text{Inf}) * (-\text{Inf}) = \text{Inf}$, $\text{Inf} * x = \text{sign}(x) * \text{Inf}$, $\text{Inf} * 0 = \text{NaN}$, $\text{NaN} * x = \text{NaN}$.

Warning

Comparison against NaN is always false : `if x < 0 or x > 1: raise` lets a NaN through, while `if not (x >= 0 and x <= 1): raise` rejects it.

Warning

Danger zone

- **Transcendental operations** (sin, cos, atan, exp, log) are much slower (50-300 clock cycles) and less accurate.
- **Fused Multiply-Add** (FMA) computes `a*b+c` directly (faster, one rounding step), but now there are two ways to do the same thing (e.g. clang-14 inserting FMAs can make `(u*600000) - int(u*600000)` negative).
- **Compiler optimizations** may violate expected IEEE 754 rounding behavior.
- **Denormalized numbers** are often very slow and cause surprises. Can turn them off.
- Careful when converting floats to integers, and vice versa.
- Arithmetic transformations that are "pointless" on pencil and paper can make a huge accuracy difference.

Types of Errors

Example

Catastrophic numerical errors : Ariane 5 rocket failure (June 14, 1996); Patriot missile failure (1991, Dhahran, Saudi Arabia) : *rounding error accumulation*, the battery failed to intercept a missile, 28 people died, 96 injured.

Definition

MODEL ERROR : the mathematical model is only an approximation (computing the surface of the earth with $A = 4\pi r^2$, but the earth is not a sphere).

Definition

DISCRETIZATION ERROR : e.g. a fluid simulation represented at different spatial resolutions (50 : bake 3 minutes; 400 : bake 13 hours). Effectively a type of model error.

Definition

TRUNCATION ERROR : happens when we approximate an infinite calculation with something finite.

$$\sum_{i=1}^{10} \frac{1}{3^i} \approx 0.499991532, \qquad \sum_{i=1}^{\infty} \frac{1}{3^i} = 0.5$$

Definition

ROUNDING ERROR

$$\sum_{k=1}^{\infty} \frac{1}{k} = 15.403683\ ?!?!$$

The series value is infinite but *converges* in IEEE-754 arithmetic!

Example

The issue here (8 mantissa bits after the point) : adding $3.14 = +2^1\, 1.10010010_2$ to $1024 = +2^{10}\, 1.00000000_2$ requires shifting 3.14 to align the exponents ($+2^{10}\, 0.00000001100100010_2$) : the low bits are **dropped** (rounding error). Once the sum is large, the small terms $1/k$ are dropped entirely.

Catastrophic Cancellation

Definition

CATASTROPHIC CANCELLATION
The especially dangerous case $a - b$ with $a \approx b$ (also $a + b$ with $a \approx -b$). Can happen even if the subtraction is done without error : it "reveals" the imprecision of the operands.

Example

8 mantissa bits after the point :

$$3.140 \approx +2^1\, 1.10010010_2$$
$$3.148 = +2^1\, 1.10010011_2$$
$$3.140 - 3.148 = -2^1\, 0.00000001_2 = -2^{-7}\, 1.0_2$$

The exponent must be adjusted to re-establish the implicit leading 1. Not a wrong answer given the approximations, but only "1 bit of information" is left from much more accurate inputs.

Example

$$f(x) = \frac{1 - \cos x}{x^2}$$

$$f(1.2 \cdot 10^{-5}) = \frac{1 - 0.9999999999}{1.44 \cdot 10^{-10}} = \frac{0.0000000001}{1.44 \cdot 10^{-10}} \approx 0.694$$

The last digit is the only bit of accuracy left. BUT one can show that $f(x) < 1/2$ for $x > 0$.

Quantifying Error

Definition

ABSOLUTE ERROR : |true value − approximate value| (e.g. 2 cm ± 0.1 cm).
RELATIVE ERROR : (absolute error)/|true value| (e.g. 2 cm ± 0.1%). Other notation : (true value)(1 ± relative error).

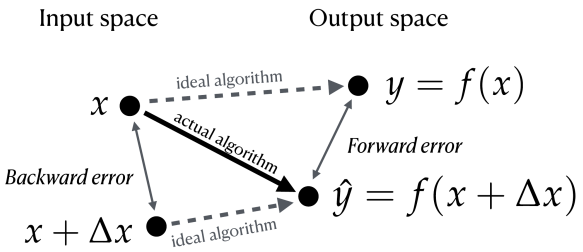
Warning

The error is generally unknown.

Definition

FORWARD AND BACKWARD ERROR
The actual algorithm maps x to $\hat{y}$ instead of $y = f(x)$; $\hat{y}$ is what the ideal algorithm returns for a perturbed input : $\hat{y} = f(x + \Delta x)$.

- **Forward error** : the distance between y and $\hat{y}$ (output space).
- **Backward error** : Δx, the distance between x and $x + \Delta x$ (input space).



Example

$f(x) := \sqrt{x}$, $y = f(x)$, $\hat{y} = f_{\text{IEEE754}}(x)$:

$$\text{forward error} = y - \hat{y}$$
$$\hat{x} = f^{-1}(\hat{y}) = \hat{y}^2$$
$$\text{backward error} = x - (f_{\text{IEEE754}}(x))^2$$

Key formulas

Conditioning of numerical problems

$$\text{Condition number : ratio } \frac{\text{forward error}}{\text{backward error}}$$

Weekly Summary

Summary

- Numerical methods = computer implementation of linear algebra and analysis, over many layers down to the processor.
- Integers, fixed point, rationals, **floating point** : sign, exponent, mantissa (implicit leading 1).
- **IEEE 754** : signed zero, infinities, NaN, denormals, round-to-nearest-tie-to-even, correctly rounded elementary operations.
- Errors : model, discretization, truncation, **rounding**; **catastrophic cancellation**.
- Absolute vs relative error; forward vs backward error; condition number = forward / backward.